

Hyperfinance Agent

An Agentic Accounting System with Deterministic Financial Engines
and Bounded Human Authority

Team hyper.*

September 19, 2026

Abstract

General-purpose LLM agents can read financial evidence, but left alone they produce narrative conclusions that cannot be audited, replayed, or safely authorized. Hyperfinance Agent separates *investigation* from *calculation* and *authorization*: agents retrieve evidence and propose work, deterministic engines recompute every amount and enforce blocking conditions, and only verified humans can attest evidence or approve outcomes. Around this core we built a voice-first workspace where a persistent agent workforce investigates anomalies over a company’s real data surface, raises structured concerns to humans, converts completed work into governed reusable skills, and renders live financial artifacts. We report honest worker-baseline measurements on four public accounting benchmarks—Invoice Sandbox (18/18 customer totals, \$0.00 aggregate error), DABstep (7/10), BenchRec (95.8% precision, 91.2% recall), and APEX (70/89 rubric criteria under a GPT-6 Astra judge)—and we are explicit that these measure the isolated agent harness, not yet the full backend-controlled system.

1 Why this exists

Accounting work done by an unconstrained LLM fails in a specific way: it writes prose. “The invoice looks fine” is not a payable proposal; “these match” is not a reconciliation. Amounts must be reconstructed from records, discrepancies must be proven against rules, and anything that moves money requires a human decision bound to the exact artifact being approved.

The design premise of Hyperfinance Agent is therefore **not** “a smarter agent.” It is a division of labor:

- **Agents are good at** searching messy evidence, noticing anomalies, drafting hypotheses, and doing new kinds of work.
- **Code is good at** integer arithmetic, three-way matching, cumulative credit checks, residual computation, and refusing ambiguous input.
- **Humans are good at** attesting that a record is real, and deciding whether a proposed journal or payable amount may proceed.

Every surface in the system is built to keep those roles from blurring. Agents cannot approve. Engines cannot guess. Humans approve hash-bound artifacts, not vibes.

2 System architecture

2.1 Interfaces and orchestration

The user-facing surface is a Next.js workspace (the “atrium”) with a voice agent (Deepgram listen–think–speak with a configurable managed think model; the code defaults to GPT-4o mini) and

*Built at HackMIT 2026. Repository: github.com/MatthewKim323/hyper.

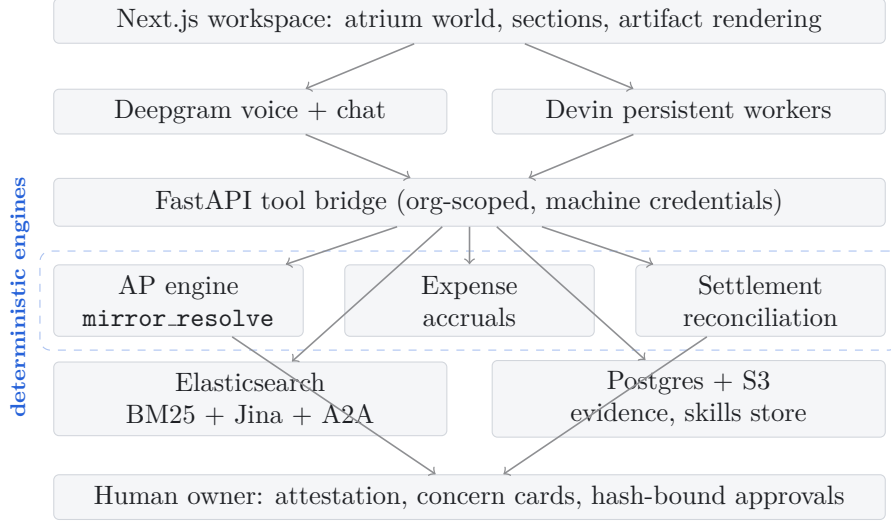


Figure 1: Layered architecture. Agents investigate through a bounded tool bridge; engines compute; humans authorize.

dashboard sections for cases, evidence, activity, review, and benchmarks. Behind it, a FastAPI backend runs a durable dispatcher that manages persistent Devin workers—at most two concurrent cloud sessions per organization, under cumulative session and ACU budgets.

Agents never hold user credentials. Each worker session receives a scoped machine token (`APP_AGENT_TOKEN`, only its hash stored) and reaches the backend exclusively through an authenticated tool bridge: `GET /agents/tool-definitions` and `POST /agents/tools`. Workers cannot delegate, cannot modify cases, cannot respond as the user, and cannot touch other organizations. Every delegation, event, and case update is recorded transactionally with request-key idempotency.

2.2 Evidence and retrieval

Connectors (Gmail, Drive, Ramp, Plaid) ingest read-only source material into Postgres/S3, indexed into Elasticsearch with hybrid retrieval (BM25 + semantic embeddings via Jina, reciprocal-rank fusion, optional Jina reranking). Beyond search, the backend delegates *investigations* to Elastic Agent Builder over the A2A protocol: the Elastic agent iteratively discovers related records through bounded ES—QL tools whose organization constraint is fixed in the tool definition, not supplied by the model. Returned findings are validated by our backend—chunk ownership, current source status, citation integrity—before becoming concerns.

Every active source also feeds a **company knowledge graph** held in Postgres beside the records it is derived from: typed entities (vendors, customers, employees, invoices, orders, agreements, payments, journals, accounts), relationships that each cite the source row asserting them, and a record of which entities every evidence chunk mentions. It is built deterministically during indexing and is rebuildable; no model writes to it. Declared foreign keys become edges at full confidence, untyped keys are typed by identifier prefix, and free text is linked by exact identifier or by a name only one entity uses. Identity is never merged: `po 62` is returned as a *candidate* for `P0-00062`, because the fixtures reuse identifiers across unrelated companies and a silent merge would be a wrong payment. Search reads the entities a question names, walks two recorded hops, and adds an exact-match retriever over those entity IDs to the fusion, so a question about a vendor reaches the documents that cite only that vendor’s order or receipt numbers. Agents get four cited graph

tools (resolve, explore, shortest path, evidence for an entity), and traversal never passes through general-ledger accounts, since “both were paid from the operating account” explains nothing.

2.3 Deterministic engines

The product’s core is a set of deterministic engines that accept only strict, normalized evidence and return exact results or explicit blockers:

Accounts payable (`/mirror_resolve + /accounting API`). Three-way matching across invoice, purchase order, receipt, agreement, and credit records, all in integer minor units (USD/EUR/GBP, no FX). Analysis requires invoice-minus-verified-credits to agree with received-quantity-at-supported-price; a price credit cannot settle a quantity exception. Proposals are hash-bound drafts validated by code (`engine:deterministic_validator`); approval is a separate owner action bound to the exact result hash, and replacing a backing source invalidates dependent proposals.

Expense accruals (`/accounting/accruals`). From three immutable packets—contract/rate/account mapping, deliveries, and the complete posted ledger—the engine filters at month-end cutoff, computes quantity × rate, subtracts net posted expense including validated reversals, and emits a balanced journal proposal plus next-day reversal. It stays unposted until owner approval; subsequent posting, reversal, and invoice observations are tracked as lifecycle evidence.

Settlement reconciliation (`/accounting/settlements`). Proves a processor batch’s arithmetic (sales, refunds, fees, reserves → payout) and its bank-statement match independently, with identity, duplicate, coverage, and completeness checks. Content-hashed runs are durable; verified bank transactions cannot be reused across runs.

Journal posting (`/accounting/journals`). An append-only ledger where approved work actually lands: the agent posts the *stored* journal of an owner-approved accrual (lines come from the persisted result, never agent-supplied amounts), manual drafts require owner approval then owner posting, and corrections exist only as reversal entries referencing the original. A trial-balance tool recomputes exact per-account debit/credit/net totals for any date and currency.

Anomaly detection (`/accounting/anomalies`). Deterministic scanners replace scripted incidents: duplicate invoices, vendor bank-detail changes, invoice–PO price variance, unmatched invoices, and settlement residuals become persisted findings with evidence citations. Findings escalate into real concern cards for owner decision; dismissal is a two-phase proposal the owner must confirm.

Evidence adapters (`/accounting/adapters`). Stripe and Adyen shaped exports and generic bank rows are normalized into the strict packets the settlement engine consumes. Every unmapped row is reported with a named reason; nothing is silently dropped or guessed.

Across all of these: unsupported fields are rejected rather than discarded, uncertain evidence blocks rather than guesses, and no agent route can approve, post a manual journal, reverse an entry, or move money.

2.4 Human authority

Three distinct checkpoints keep authority with people:

- **Attestation.** Only an organization owner can mark a structured source row as verified evidence (`OWNER_VERIFIED`, verifier `human:<clerk-subject>`). An LLM cannot promote its own extraction to trusted input.
- **Concern cards.** When agents hit ambiguity or missing evidence they raise concerns; Jev generates a card with three checked options plus a custom response. Cards never approve journals—they resolve questions.

- **Hash-bound approval.** Every approvable artifact (payable proposal, accrual, reconciliation) is approved against its exact result hash. There is no agent tool for approval at all.

2.5 Learned skills

Completed workflows become procedural memory. A governed extraction path (`/accounting/skills/extract`) turns a completed task or case into a draft package only when it carries live evidence citations and a tests resource; lineage rows record where each skill came from. The lifecycle: discover existing skill → research gaps → implement and execute in the worker’s isolated workspace → save execution logs → save an immutable draft package (SKILL.md generated by the service; scripts/tests/references as integrity-checked resources) → record a run report → owner reviews the actual tests and evidence and activates → retrieval on similar tasks. Failure reports quarantine active versions immediately. Skills are org-scoped packages in Postgres metadata + S3 storage, indexed summary-only in Elasticsearch with a SQL fallback. The backend never executes skill code; self-reported passes are labeled as such, and activation is the independent promotion gate.

2.6 Artifacts

For live financial visuals, agents call a composition tool backed by json-render: Jev emits a fixed three-node spec (chart type, points typed actual/projected, decimal strings) which the frontend renders with bklit charts—fast enough for in-conversation projections.

3 Evaluation

3.1 Method

We measured **isolated worker baselines** on four public accounting benchmarks: each run uploads inputs only to a fresh Devin cloud session with no Hyperfinance Agent backend tools, no answer keys, and an ACU cap; grading keys and scorers stay local; prompts and input archives are hash-pinned. We report every run, including a mid-run ACU suspension.

3.2 Results

Benchmark	Workload	Result	Grading
Invoice Sandbox	112 PDFs, 18 customers	18/18 totals, \$0.00 error	pinned native scorer
DABstep (dev)	10 questions, 138k rows	7/10 (6/7 hard passed)	pinned native scorer
BenchRec v3	32,048 bank rows	95.8% precision / 91.2% recall	custom metrics
APEX (dev)	10 month-end tasks	70/89 criteria, 0.710 macro	GPT-6 Astra judge

BenchRec matchers execute in a locked-down container (no network, read-only, pinned image) before scoring. DABstep passed 1/3 easy and 6/7 hard tasks in the original run. The original APEX worker hit its 10-ACU cap before emitting output and required a follow-up message. Its initial GPT-4.1 grade (71/89) contained an internally inconsistent numerical-tolerance verdict. A fresh `gpt-6-astra` assessment of the unchanged answers yields 70/89 criteria and 71.0% macro task score. The old grades are preserved; changing the judge is not an agent improvement. This remains a provisional local model judgment, not expert or official grading.

3.3 Same-data baselines and repeated trials

The BenchRec version-3 distribution includes `MatcherByChatGPT_submission.csv`. We scored it locally with the same exact-allocation metric and all 32,048 expected bank IDs. Its zero-or-one-element prediction arrays were normalized without selecting among guesses. The filename does not establish the underlying model, prompt or compute budget. It is a distributed reference submission, not a verified frontier-model baseline.

BenchRec method	Precision	Recall	Coverage
Distributed reference	95.20%	62.20%	64.90%
Original Devin worker	95.83%	91.22%	94.56%
Verified-v2, mean of 3 trials	96.23%	88.72%	91.58%
Unique exact-total rules	99.51%	46.48%	46.40%
Exact-reference conservation rules	99.86%	8.70%	8.66%

Verified-v2 adds source-grounded checklists, executable calculations and cross-checks, allocation conservation, interpretation of the supplied manuals, and final-output review. Three fresh trials per benchmark were authorized under 60 ACU of total configured caps. Inputs are unchanged; no prior answers or grading feedback are supplied. BenchRec gains 0.40 precision percentage points on average but loses 2.50 recall points; none of its three trials reaches 99.8% precision. DABstep scores 7/10, 6/10 and 7/10 (66.67% mean), versus the original 70%: no improvement is demonstrated. All three APEX candidate sessions stopped at their 10-ACU caps without complete outputs: completion is 0/3 and all three attempts remain ungraded. They were not resumed or granted higher caps; this differs from the original baseline’s completion nudge and prevents a clean candidate accuracy comparison.

A separate post-hoc conservation gate on the original BenchRec assignments raises precision only to 95.92% while reducing recall to 84.92%. This negative result shows that balanced amounts are insufficient to establish identity. The rules-only precision result likewise cannot be presented without its low recall. Source-only DABstep diagnostics identified count-versus-volume denominators, null wildcard handling, and unsupported fine thresholds as concrete targets for executable checks rather than further prompt expansion.

3.4 Retrieval with the knowledge graph

We measured evidence retrieval on the synthetic Meridian company: 612 draft emails, chats, notes and contract excerpts among 115,348 ledger rows, first against a local Elasticsearch 9.3 with ELSER v2 embeddings on documents and no reranker. The answer key is the list of record identifiers each document was authored from, which is never indexed. 172 templated questions fall in three families: an identifier the documents also contain, a counterparty name only, and an identifier *no document mentions* (a payment, a journal entry) that only a recorded relationship connects to the correspondence. That last family is keyed by joining the raw exports, not by asking the graph. Scores are over the top ten documents.

Retrievers	Recall@10	Unwritten-ID recall@10	MRR@10	nDCG@10	p50
BM25	0.546	0.252	0.668	0.544	245 ms
BM25 + ELSER	0.598	0.248	0.734	0.619	269 ms
BM25 + graph	0.930	0.852	0.946	0.892	293 ms
BM25 + ELSER + graph	0.937	0.861	0.971	0.906	294 ms

Embeddings alone move little here, because the documents that matter cite VEN-002, not the vendor’s name. An identifier nobody wrote down is nearly unfindable by text and is recovered by one or two recorded hops. The first fusion treated BM25, embeddings and the graph retriever as three equal voters and scored 0.735 recall@10, *below* the graph with no embeddings at all: two text voters outvoted an exact match. Text is now fused first and the graph fused against the result.

The same questions were then asked on **Elastic Cloud Serverless** with **Jina** models through-out: `.jina-embeddings-v5-text-small` behind `semantic_text` and `.jina-reranker-v3.5` for reranking, both preconfigured on the project.

Serverless, Jina models	Recall@10	Unwritten-ID recall@10	MRR@10	nDCG@10	
Jina v5 (BM25 + embeddings)	0.533	0.233	0.702	0.550	44
Jina v5 + Jina reranker	0.663	0.287	0.744	0.678	103
Jina v5 + graph	0.936	0.861	0.919	0.884	54
Jina v5 + graph, reranker over the fused list	0.770	0.377	0.764	0.749	109
Jina v5 + graph, reranker on text only	0.944	0.864	0.940	0.907	109

Two results, neither of which we expected to write. First, the embedding model barely matters on this corpus: Jina and ELSER with the graph land within a thousandth of each other, because recall here comes from recorded relationships. Second, the reranker placed where every tutorial places it, over the final fused list, made retrieval *worse*, and by a wide margin on the questions only a relationship can answer. A cross-encoder scores wording, and a document that is relevant because its invoice was settled by the payment in the question shares no wording with it, so reranking rescored the graph’s contribution away. The reranker now judges the text retrievers alone, and the graph retriever votes against the reranked text. That is the best configuration measured, it keeps what reranking is good at (it sharpens the top of the list when the question and the document do share words), and it costs roughly twice the latency. The production deployment was running the first shape until this measurement.

The limits are plain: the documents are synthetic drafts that repeat their identifiers, which flatters every lexical mode; the questions are templated, so this measures entity and relationship recall and says nothing about paraphrase; relevance is binary and document-level. The questions are asked again whenever retrieval code changes and the scores are appended to the timeline below.

3.5 A continuous adversarial loop, measured over time

The product-level test the previous section called for now runs unattended. An adversary spawns accounts-payable exceptions into a sandbox organization and role-plays the supplier and the internal desks; a worker (`gpt-5.6-terra`) resolves each case through the real tool bridge and deterministic engine; grading reads persisted engine state against a private expectation, never the worker’s prose. After each graded case the worker may write one short lesson, which later sessions receive. Difficulty adapts: three correct results raise the tier and a miss lowers it. A recorder runs beside the loop, rebuilds every series from graded cases on each read, and stamps suite and retrieval snapshots with the commit, so the figures below are generated from the database at build time (2026-09-20 14:00 UTC, commit `cd6cf1e`; backend suite 483 passed, 0 failed).

The experiment that isolates learning is a twin: every case is mirrored, by family and tier, into a control organization whose worker is identical but receives no lessons.

Tiers one to four name their own diagnosis in the case title and are saturated in both arms; they show a competent worker, not learning. The trap tiers are the only ones where every engine check passes while the right answer is to hold (an approved contact has said not to pay, a verified credit

Cases	Arm	Pairs	Correct	Wrong releases	Requests	Median	Cost/case
Trap tiers (5+)	with memory	191	186/191 (97.4%)	3	1.46	57.1 s	\$0.028
Trap tiers (5+)	without memory	191	105/191 (55.0%)	86	0.69	61.5 s	\$0.016
All tiers	with memory	302	297/302 (98.3%)	3	1.61	45.7 s	\$0.029
All tiers	without memory	302	215/302 (71.2%)	86	1.05	55.5 s	\$0.018

was withdrawn, an invoice was superseded), so they are the only place a wrong release can occur. There the control arm mostly read the warning and paid anyway. The arm with memory missed each new kind of trap the first time it met it and not again:

Trap family	Cases	Missed on encounter
<code>internal_hold</code>	31	1
<code>withdrawn_credit</code>	26	1
<code>already_paid</code>	26	3
<code>superseded_invoice</code>	24	none
<code>spoofed_release</code>	19	none
<code>internal_release</code>	18	none
<code>unrelated_wire</code>	15	none
<code>goods_returned</code>	13	none
<code>short_credit</code>	9	none
<code>cleared_hold</code>	9	none
<code>misdirected_hold</code>	7	none

Memory buys this by checking: it sends more requests per trap case and costs more per case at list prices. It also finishes released cases sooner, though the median time on a hold is set by the grader’s fixed wait and not by the worker. We report the cost beside the accuracy rather than instead of it.

What these numbers are not. This is a **development** result: one author wrote the scenarios, the worker prompt and the grader, and nothing is held out. It is one continuous run with simulated counterparties, and the twins are matched by family and tier, not by invoice, so each arm sees its own amounts. Three sandbox defects were found during the run and cut from every figure rather than averaged in: cases created before two fixes (ignored follow-ups, unusable record identifiers) are excluded outright, and 10 wrong releases were dropped because the opening warning was delivered about a second late and the worker was graded for ignoring a message it had never been shown. One of those cuts reversed an early reading in which memory appeared to *hurt*. Rates are never computed under twenty cases. The honest next step is a trap battery written by someone other than the prompt’s author.

3.6 Published comparisons and limitations

The [DABstep launch report](#) gives historical hard-test baselines, including approximately 16% for o3-mini. These are neither current frontier results nor comparable to our ten-question development split. The [APEX leaderboard](#) evaluates 160 private tasks across ten worlds; its current headline table reports 61.0% for Fable 5.1 and 60.0% for GPT-6 Astra. Different inputs, task isolation, harnesses and graders prohibit claiming that our public-development scores beat those systems.

Repeated public-fixture trials measure variation, not held-out generalization. The original baseline has one run; provider-managed worker models are not pinned. Repeated questions are correlated, not additional independent tasks. Missing outputs stay in the attempted denominator.

No run uses Hyperfinance Agent’s accounting APIs or learned skills; the next product-level comparison must exercise that real tool bridge. Cloud answer-key isolation is instruction-based, not a firewall. New judge caches bind model, protocol, rubric and answer hashes and never overwrite legacy grades. Current checks passed 22 focused runner/scorer tests and all 323 evaluation tests.

4 Current state and limitations

Implemented and test-covered: the full tool bridge and dispatcher, all three engines with owner-only approval paths, the learned-skill lifecycle with quarantine, Elastic A2A delegation with backend citation validation, the knowledge graph and graph-widened retrieval, the adversarial exception loop with its memory-off twin, the benchmark timeline recorder, concern cards, artifacts, and the benchmark harness itself.

Not yet proven live: Clerk-authenticated onboarding end-to-end, the Elastic A2A path against a provisioned agent (credentials pending), a live Devin session through a public tunnel, and Postgres-backed aggregates outside the compose stack. BenchRec precision (95.8%) shows real headroom on high-precision matching. The engines deliberately cover narrow ground: whole-unit records, USD/EUR/GBP, no taxes, no FX, no split deposits, no external posting.

5 Conclusion

Hyperfinance Agent treats accounting agents as investigators operating under engineering controls, not as oracles. The measurable claims today are a competent isolated worker, a retrieval layer whose recall comes from recorded relationships rather than from embeddings, and, on a development set, a worker that stops repeating a mistake once it has written it down. The architectural claim—which the held-out battery is needed to test—is that binding agent proposals to deterministic engines and hash-bound human approval converts fluent output into auditable financial work.